Final Project Requirements, Questions and Architecture

Final Project Definition

The project you are designing here is **your** final project — either one of the ideas on the **Final Project Specifications** page, or a project idea the instructor has approved. Read that page end to end (overview, required showcase sections, rubric, and the idea you picked) before you start this exercise.

Assignment Overview

Anyone can build an agentic system with Claude Code or Open Code. *Can you build one that is reliable, safe, cost effective and meets all the requirements every time it's run?* It's time to put on your AI architect hat and address LLM/RAG shortcomings with a good agentic AI design.

Rubric

This is a 500 point exercise. It's an assessment of how much you have learned up to this point. Please take it seriously. It's as important as the project output. You should complete this Quarto markdown file (.qmd) and turn it in along (as .qmd) with a quarto rendered PDF version (.pdf). Add `format: pdf` to the yaml header of this file and then run `quarto render yourfilename.qmd` to build the PDF.

Important Note

This work must be done 100% without assistance from any AI. I need to know that *YOU* have learned the material. I know that Claude knows it. **Any use of AI will be easily detected and will be treated as an Honor Code violation. Please take this seriously.**

Introduction

Unlike the mid-term, there is no single specification for everyone — so before you list requirements, tell me what you are building. Name the project you chose: one of the ideas on the Final Project Specifications page, **or** a project the instructor has approved (say so, and when it was approved).

Then write a few paragraphs (in prose) explaining what you are trying to accomplish. Who is the user? What problem do they have today, and what will they be able to do with your solution that they cannot do now? What is in scope and — just as important — what is out of scope? What does a successful run look like? Finally, make the case for why this problem **needs a multi-agent design** rather than a single prompt or a script.

The project we have chosen is 35.4.12 Sports: Opponent Tendencies Analyst from the list of ideas on the Final Project Specifications page.

The goal of this project is to provide a coaching staff or game-preparation analyst with a single, evidence-based opponent tendencies report for an upcoming NFL game. More often than not, play-by-play data and qualitative context from beat reporters and coaches' quotes are not brought together in one place.

We want to create a tool that allows a defensive coordinator, analytics staff member, or fan analyst to better prepare for an upcoming opponent. Currently, this research is often done in separate pieces. Analysts query or examine play-by-play data to identify what opponents tend to do in certain situations, such as third-and-long, fourth down in opponent territory, or first-and-goal. They then listen to opposing teams' postgame press conferences to gather additional insight and investigate what other reputable analysts are saying online. With our solution, we hope to create a single platform where users can access all of this important information and receive a user-friendly report outlining an opponent's tendencies.

For the scope of this project, we plan to focus on one opponent per run, regular-season games from the current season, publicly available data and reporting, and a written report supported by sourced evidence. Some examples of what would be considered out of scope include predicting game outcomes or betting lines, providing real-time in-game analysis, guaranteeing injury statuses, and recommending specific game plans. The tool is designed to describe what an opponent tends to do and why those tendencies might be changing; it is not designed to tell the user what actions to take.

A successful run of this application would follow a straightforward process. The user selects an opponent from a dropdown menu of available teams and receives a comprehensive report on that team's tendencies within a few minutes. The report will contain both quantitative and qualitative sections, providing relevant information in formats such as statistical tables, summaries of tendencies, scheme notes, and quotes. All statistics should be recalculated from stored data, and every quote should include a citation to ensure traceability.

This problem requires a multi-agent system rather than a single prompt or script for several reasons. First, the data sources are very different from one another. Play-by-play data is structured, while articles and transcripts are largely unstructured and require search, retrieval, and reading comprehension. Distributing these tasks among multiple specialized agents should improve the results gathered from our separate data sources. Second, the work is naturally parallel and independent. A statistics agent, a beat-reporter and analysis agent, and a coach-quote agent can each gather evidence simultaneously using different tools. Each agent can also be independently tuned and evaluated. Third, the project requires a fusion step. A synthesis agent can take the results gathered by the other agents, compare their findings, and identify potential contradictions or inconsistencies. This approach allows us to bring together different perspectives and evidence sources in a way that a single prompt might otherwise overlook or blur.

Requirements

List the requirements for **your** project, based on the Final Project Specifications (the overview, required showcase sections, rubric, and the description of the idea you chose — or your instructor-approved proposal) and on the design you are proposing in your Introduction. List as many as you can (20 is not meaningful here). This is a critical step! Any person or LLM should be able to fully understand **all** requirements for the application from this list.

Number every requirement — the later sections refer back to them by number.

1. The solution should provide a user-friendly user interface in Python Streamlit.
2. The solution should get NFL play-by-play data for the 2026 season from nflverse and store it in a local JSON database.
3. The user should be able to refresh and update the local database with the latest and most up-to-date 2026 season from nflverse. It should not create duplicates by using the game ID and play ID.
4. The solution should let you select an opponent from a dropdown of all NFL teams in alphabetical order. Also, it should let you choose which regular season weeks the user wishes to include. Preseason, future weeks, and postseason should not be included.
5. The solution should show the important information such as the opponent, games included, date, scores, and data used, so that it is easy to understand.
6. The solution should retrieve only relevant written beat-reporter articles, film breakdowns, and post-game coach quotes for the selected opponent from the stored index.
7. The solution should store articles, breakdowns, coach quotes, and any relevant metadata like author, publisher, date, link, and the game it refers to, in a database for later use or retrieval.

8. A vector RAG index should be created for the articles, breakdowns, and coach quotes using ChromaDB so the system can retrieve evidence for each opponent. The index should be saved for reuse and only rebuilt when the source data changes. The same embedding model should be used for indexing and searching.

9. After selecting an opponent, the user should be able to click a button to build an opponent tendencies report backed by evidence.

10. The solution should use four agents to build the report. One analyzes play-by-play statistics, one reviews beat-reporter coverage and film breakdowns, one reviews coach quotes, and the last combines all findings. The first three agents should work at the same time. Python checks calculations, format, and source IDs. An LLM call checks claims against sources and corrects unsupported claims before showing them. Prompt optimization is done offline at design time and is not a runtime agent. The follow-up chat reuses the synthesis agent.

11. The statistics agent should use Python tools to calculate the opponent's tendencies such as the run rate, pass rates, yards per play, and third-down conversion rates. It should compare situations based on down, yards needed for a first down, and field position. It should be consistent and show the number of plays included.

12. The beat-reporter agent should retrieve only relevant articles and written film breakdowns within this season of football. It should get it only from known sports sites and trusted reporters and analysts. It should find opponent tendencies, formations, play calling, and recent changes. All findings should be supported.

13. The coach-quote agent should retrieve only relevant post-game statements from coaches of the selected team within this season of football. It should look for explanations of the coaches' approach, adjustments, and performance. Direct quotes should keep the same wording and back it up with which coach said it, the date, and the source.

14. The synthesis agent should combine all 3 of the agents' findings into one report. It should explain where the information agrees, disagrees, and what's missing. Its format should be clear, easy to understand, and consistent.

15. A coach's words should not be treated as proof of actual tendencies. It should identify changes in the opponent's tendencies when enough data is available. It should show which games or weeks are being compared.

16. The solution should show a written report with quantitative and qualitative sections. The quantitative section should include tables or summaries of tendencies. The qualitative section should include scheme notes, reporting, and coach quotes.

17. The solution should allow the user to follow up on the initial report by asking about a particular situation, requesting supporting evidence, or asking for an explanation. The selected opponent, games, report, and evidence should be kept in session state between turns.

18. The solution should optimize the original prompt using LLM prompt optimization. The rewritten prompt should keep all important information.

19. The solution should verify all facts presented through direct grounding in the source materials. The user should be able to trace back factual claims to the original source text or the stored information.

20. The solution should be reliable and testable. With the same data, opponent, weeks, and prompt version, the statistics table must be identical on every run and the quotes must be 100% traceable on every run. Also, over 5 runs, at least 80% of the top 5 narrative tendencies must appear in at least 4 of the 5 runs, with no contradictions in direction between runs.

21. The solution should validate data responses and retrieval results before allowing the model to use them as evidence. Also, code should check that every game, play, article, and quote exists in the source data.

22. The solution should clearly report no results, invalid responses, or failed tool calls instead of making up missing information. If relevant articles, breakdowns, or coach quotes are unavailable it should report what is missing as a warning. It should then continue with the information that is supported, following the degraded and hard-failure rules.

23. The solution should treat retrieved information as data, not as instructions for the model to follow.

24. The solution should use deterministic code for calculations, sorting, filtering, and comparing opponent statistics instead of asking the LLM. Also, Python code should check for duplicate plays and missing values. If missing data affecting calculations exists, it should explain and not make up a value.

25. The solution should return each agent's findings in a consistent structured format. Each agent's output has to include the opponent, games, weeks chosen, tendencies, observations, supporting evidence, and limitations.

26. The solution should make sure sources line up with the opponent and weeks chosen. Reporting from earlier in the 2026 season can be labeled as background, but shouldn't be evidence of tendencies for the weeks chosen. Reporting from previous seasons should not be used at all.

27. The solution should flag it or ask for human review when the available evidence is too weak to support the opponent's tendencies claim. Things like small samples or missing sources should be reported.

28. The solution should make sure to focus on describing what the opponent tends to do. Predicting game outcomes or betting lines, in-game real-time use, injury-status guarantees, and recommending game plans are out of scope.

29. The chatbot and agents should use an LLM through the VT ARC OpenAI-compatible endpoint. Each agent should have a narrow-focused constitutional prompt that states its objective and rules to follow.

30. The solution should produce a report within a few minutes by using stored data and show progress while running. Each agent has its own timeout. If the beat-reporter or coach-quote agent fails or times out, the report is still produced with that section marked unavailable. If the statistics agent fails or the overall time limit is exceeded, no report is produced and the app explains what failed.

LLM / RAG Issues

Referring back to the lectures on LLM and RAG — and, now that you have seen them, on multi-agent systems and anti-patterns — what are the specific issues that you must overcome with good agentic AI design/architecture/patterns **to meet the requirements**? **Make sure you point back to a specific requirement**. Don't solve the issue yet. Just identify it. You should have a fair number of these (between 5 and 20).

1. For requirements 2, 3, 6, 12 and 13, the LLM has a brain-in-a-box limitation. The LLM can only produce tokens and cannot retrieve information from nflverse, articles, film breakdowns, or coach quotes by itself.
2. For requirements 2, 3, 12, 13, and 26, the LLM has frozen knowledge. The model's training cutoff means it is not up to date. It does not know the latest NFL games or reporting.
3. For requirements 12, 13, 19, 21, and 22, the LLM could hallucinate facts about an opponent. It could invent tendencies, formations, quotes, or sources that sound correct but aren't. It has no independent external ground truth for validation.
4. For requirements 14 and 20, LLM responses are non-deterministic, meaning the same prompt could produce very different opponent's tendencies report if run multiple times. Also, the model could make the same mistake across repeated runs.
5. For requirements 11, 15, and 24, the LLM is bad at exact computation. LLMs are unreliable for math and could incorrectly calculate run rates, pass rates, yards per play, or third-down conversion rates.
6. For requirements 7, 8, 12, and 13, poor chunking could prevent RAG from retrieving important evidence like articles or quotes. It can answer incorrectly because retrieval missed a relevant part of the answer.
7. For requirements 6, 8, 12, and 26, semantic similarity does not always mean relevance. An article may have football knowledge but it doesn't prove it is an offensive tendency for that opponent or game.
8. For requirements 14, 16, and 19, lost-in-the-middle could cause the LLM to not see important statistics or article evidence because it got buried in the middle of a long prompt.

9. For requirements 17 and 26, context rot could cause the chatbot's answers to become less trustworthy as the conversation gets longer. Also, recency bias could cause it to focus too much on the latest question and overlook things such as the weeks chosen.
10. For requirements 17, 23, and 29, prompt injection could occur if a user or retrieved document has text that looks similar to instructions. The model could follow those instructions instead of its own rules.
11. For requirements 17, 27, and 28, sycophancy could cause the chatbot to be agreeable to everything. It could confirm that an offense always passes in a certain situation even when not supported.
12. For requirements 8, 10, 14, and 30, latency is an issue because LLMs generate tokens that pile up one at a time, and every retrieval step or extra LLM call makes the user wait longer.
13. For requirements 10, 17, and 30, cost potentially becomes a problem because LLMs consume tokens. Multiple agents are more expensive; they use their own prompts and evidence increasing token use. Follow-up turns send a long growing context.
14. For requirements 14, 16, 21, and 25, the LLM may return findings in an invalid format. Opponent information, weeks selected, evidence, or limitations could be missing or inconsistent between runs. This could cause the app to fail to show the results correctly.
15. For requirements 18, 20, and 26, prompt sensitivity means small changes in wording could change which evidence is retrieved and affect the final result of the opponent's tendencies.
16. For requirements 10, 14, and 25, information can be lost when being passed between multiple agents. One agent could leave out an important detail like a game or source. This means the synthesis agent would be left without the information it needs to have an accurate final report.
17. For requirements 10, 14, 19 and 21, error propagation can spread an agent's mistake. One agent could misinterpret a fact and the synthesis agent could repeat that mistake in the final report.
18. For requirements 22 and 30, an unbounded loop could occur if an agent tries to keep retrieving evidence that is missing without knowing when to stop. It could repeat the same tool calls that keep failing.

Basic Architecture

Write up a couple of paragraphs (in prose) on how you plan to design the solution. What will be the general approach. Don't solve the issues yet… just general approach. What are the major pieces, how do they connect, etc.?

25 Bonus points for including a UML sequence diagram using a mermaid diagram. For this syntax (and only this syntax), I will allow LLM usage** but you need to prompt the LLM with the sequence diagram flow, actors and steps. Include your prompt here.**

You should end your `Basic Architecture` with the level of agency you think is **earned** by this solution architecture. Go back to the beginning of the course for the various levels. **Make**

sure you give a quick summary of why it is earned.

The solution will be architected as a pipeline with four layers: data, validation, agents, and interface. The data layer has two halves. Structured play-by-play data for the current season is pulled from nflverse into a local JSON store, keyed by game ID and play ID so refreshes never create duplicates. Unstructured media data is collected from known sports sources and stored with its metadata (author, publisher, date, reference game). That text is also embedded into a persistent ChromaDB index, so the retrieval agents can pull evidence for the selected opponent and weeks. Between the data layer and the agents sits a validation layer in plain Python. It checks that games, plays, articles, and quotes actually exist in the stored sources, screens for duplicates and missing values, and confirms that sources line up with the chosen opponent and weeks before anything reaches a model. Retrieved text is passed to the agents as quoted data, never as instructions.

On top of that foundation, a central orchestrator takes the user's selection of opponent and weeks and fans out to three specialist agents that run at the same time. The statistics agent calls deterministic Python tools to compute pass rates, yards per play, and third-down conversion by down, distance, and field position, and it's job is to interpret and report those numbers rather than producing them. The beat-reporter agent and the coach-quote agent each use retrieval over the validated, opponent-filtered results. Each agent has a constitutional prompt and returns its findings in a shared schema, so the outputs are directly comparable. Those results fan in to a fourth synthesis agent, which combines them into one report and states where the evidence agrees, where it conflicts, and what is missing. It treats coach statements as context, not proof of tendencies. Python checks calculations, format, and source IDs. An LLM call checks these claims against their sources and corrects anything unsupported. This does not add another agent. The streamlit app is the frontend of this, as it handles team selection, shows progress while the agents run, renders the report sections, and keeps the opponent, report, and evidence in session state so a follow up chat can answer questions about the specific situations and trace back to their sources. All LLM calls go through the VT ARC OpenAI-compatible endpoint.

The level of agency earned by this solution architecture is a centralized, orchestrated multi-agent system where python controls the main steps of the process and the agents combine the evidence. This is because the work splits into independent jobs that need different tools: deterministic code for play-by-play statistics and retrieval for articles and coach quotes. As mentioned before, the three specialist agents can run in parallel, each with a narrow prompt and a shared output schema and a fourth synthesis agent to combine everything.

Prompt: You are a technical diagramming assistant. Produce ONE Mermaid `sequenceDiagram` showing the main runtime flow of the system below. Output only a single mermaid code block — no prose, no explanation.

System

A Streamlit app ("Opponent Tendencies Analyst"). The user picks an NFL opponent and the regular-season weeks to include. The app computes play-by-play tendencies in deterministic Python from locally stored nflverse data, retrieves supporting articles, film breakdowns, and coach quotes from a vector index, and asks an LLM (VT ARC endpoint) to interpret and synthesize the findings into one evidence-backed report with citations. Retrieved text is treated as data, not instructions.

Actors (declare exactly these, in this order)

- USR as Analyst
- UI as Streamlit UI
- ORC as Orchestrator
- VAL as Validation Layer
- STO as Data Stores (play-by-play + media + ChromaDB)
- STA as Statistics Agent
- BRA as Beat-Reporter Agent
- CQA as Coach-Quote Agent
- SYN as Synthesis Agent
- LLM as LLM Endpoint
- SES as Session State

Steps (the only ones to show)

1. USR -> UI: open the app; UI shows a team dropdown and week selection (played regular-season weeks only).
2. Optional: USR -> UI: refresh data; UI -> STO: upsert play-by-play keyed by game ID + play ID (no duplicates); STO –» UI: inserted / skipped counts.
3. USR -> UI: select opponent + weeks, then click "Build report".
4. UI -> ORC: opponent + weeks.
5. ORC -> VAL: confirm matching games, plays, articles, and quotes exist for this opponent and these weeks.
6. ORC fans out with `par` to STA, BRA, and CQA at the same time (same opponent and weeks, each with a timeout).
    - STA: deterministic Python stats (run/pass rate, yards per play, third-down rate) -> LLM interprets only.
    - BRA: retrieves articles and film breakdowns -> LLM extracts tendencies with citations.
    - CQA: retrieves post-game coach quotes -> LLM extracts verbatim quotes with coach, date, source.
7. After fan-in: ORC -> VAL: check each agent's output against the shared schema and confirm every cited item exists in the stores.

8. Use `alt` for failure handling:
    - Statistics agent fails OR overall time limit hit: UI shows a hard failure explaining what failed; no report is produced.
    - Beat-reporter or coach-quote agent fails/times out: mark that section unavailable, warn, and continue.
9. ORC -> SYN: pass validated findings; SYN -> LLM: merge into one report stating agreement, conflict, and gaps; flag weak evidence for human review; treat coach quotes as context, not proof.
10. SYN -> UI: show quantitative tables and qualitative notes with citations; UI -> SES: store opponent, weeks, report, evidence.
11. USR -> UI: ask a follow-up; UI -> SYN (via stored session state, no full re-run); SYN -> LLM answers with citations, or says the evidence does not support the premise; SYN -> UI: reply.

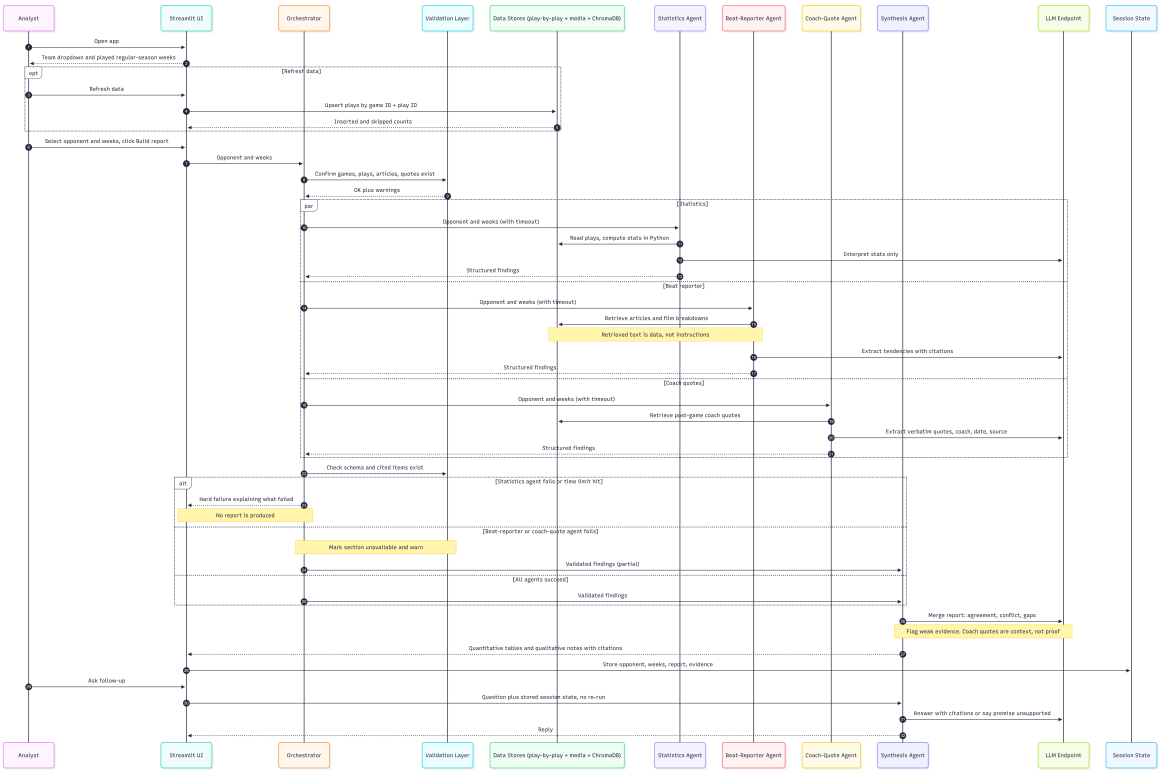


Figure 1: Sequence Diagram

```
sequenceDiagram
    autonumber
    participant USR as Analyst
```

```
    participant UI as Streamlit UI
    participant ORC as Orchestrator
    participant VAL as Validation Layer
    participant STO as Data Stores (play-by-play + media + ChromaDB)
    participant STA as Statistics Agent
    participant BRA as Beat-Reporter Agent
    participant CQA as Coach-Quote Agent
    participant SYN as Synthesis Agent
    participant LLM as LLM Endpoint
    participant SES as Session State

    USR->>UI: Open app
    UI-->>USR: Team dropdown and played regular-season weeks
    opt Refresh data
        USR->>UI: Refresh data
        UI->>STO: Upsert plays by game ID + play ID
        STO-->>UI: Inserted and skipped counts
    end
    USR->>UI: Select opponent and weeks, click Build report
    UI->>ORC: Opponent and weeks
    ORC->>VAL: Confirm games, plays, articles, quotes exist
    VAL-->>ORC: OK plus warnings
    par Statistics
        ORC->>STA: Opponent and weeks (with timeout)
        STA->>STO: Read plays, compute stats in Python
        STA->>LLM: Interpret stats only
        STA-->>ORC: Structured findings
    and Beat reporter
        ORC->>BRA: Opponent and weeks (with timeout)
        BRA->>STO: Retrieve articles and film breakdowns
        Note over BRA,STO: Retrieved text is data, not instructions
        BRA->>LLM: Extract tendencies with citations
        BRA-->>ORC: Structured findings
    and Coach quotes
        ORC->>CQA: Opponent and weeks (with timeout)
        CQA->>STO: Retrieve post-game coach quotes
        CQA->>LLM: Extract verbatim quotes, coach, date, source
        CQA-->>ORC: Structured findings
    end
    ORC->>VAL: Check schema and cited items exist
    alt Statistics agent fails or time limit hit
        ORC-->>UI: Hard failure explaining what failed
```

```
        Note over ORC,UI: No report is produced
    else Beat-reporter or coach-quote agent fails
        Note over ORC,VAL: Mark section unavailable and warn
        ORC->>SYN: Validated findings (partial)
    else All agents succeed
        ORC->>SYN: Validated findings
    end
    SYN->>LLM: Merge report: agreement, conflict, gaps
    Note over SYN,LLM: Flag weak evidence. Coach quotes are context, not proof
    SYN-->>UI: Quantitative tables and qualitative notes with citations
    UI->>SES: Store opponent, weeks, report, evidence
    USR->>UI: Ask follow-up
    UI->>SYN: Question plus stored session state, no re-run
    SYN->>LLM: Answer with citations or say premise unsupported
    SYN-->>UI: Reply
```

Solution Mitigation

Now for each of the LLM/RAG issues, you should give specific design patterns that we learned in class to address them. I need a short explanation of why your solution is the right one. **Make sure you refer back to the numbered issue from the previous section.**

1. For issue 1, the design pattern that addresses this issue is tool use. Workflow code calls tools to retrieve nflverse data, articles, film breakdowns, and coach quotes and returns the results to the agents. Since the model does not have to access the outside system, the tools allow it to act and reach the outside system, fixing the issue.

2. For issue 2, the design pattern that addresses frozen knowledge is grounding through retrieval. Tools update the stored 2026 NFL data and reporting to get the current information. Also, RAG retrieves the relevant passages. This is better than using training data because the sources can be updated during the season.

3. For issue 3, the design patterns that address hallucination are grounding through RAG and Chain-of-verification. RAG directly retrieves articles and quotes. Python gets the calculated statistics. Chain-of-verification checks them separately against their sources. This checks with evidence that can be backed up instead of just telling the model not to hallucinate. Claims built on small weak evidence should be flagged for human review.

4. For issue 4, the design pattern that addresses this issue is structured output. Using the same format and a low temperature helps keep the agents and the final reports consistent. Low temperature does not ensure consistent results, so repeated runs should be tested using the same information. It should compare the tendencies and format returned.

5. For issue 5, the design pattern that addresses this issue is tool use with Python calculations because it is deterministic. Python does the math, and the LLM explains the results. This is more reliable because LLMs are bad at math, and code can do the same calculations each time.

6. For issue 6, the design pattern that addresses poor chunking is grounding through retrieval. There should be tests with different strategies of chunk sizes and overlap. Then check what the retriever returns and ask whether it has what it needs. This is better than using one chunk size without checking because information can get missed.

7. For issue 7, the design pattern that addresses semantic similarity is retrieval with metadata filtering. ChromaDB should filter passages by opponent, year, and chosen weeks. The agent should check if passages support the tendencies in the report. This is better than similarity because the information in it could refer to the wrong team or dates.

8. For issue 8, the design patterns that address lost-in-the-middle are externalization and retrieval. The system should pin the opponent, weeks, and goal near the top of the context, and store the longer data or information outside of it. Also, it should only retrieve the relevant information. This is a lot better than putting every article or play into a long prompt. It makes important information easier to find.

9. For issue 9, the design patterns that address context rot are externalization and summarization. The opponent, game, report, and evidence should stay in session state and be put back into each prompt. Older turns can be summarized in order to shed tokens and keep only the information needed. This is done so important details don't get lost in the conversation.

10. For issue 10, the design pattern that addresses prompt injection is defense in depth. Use instruction hierarchy, delimiters, and provenance labels to treat information as data rather than instructions. Tool allowlists and schema validation add extra protection. No single guardrail can stop every prompt injection, so using multiple in defense in depth is the best option.

11. For issue 11, the design pattern that addresses sycophancy is tool-based verification. It needs an outside check on the answer. The chatbot should check the user's claim against actual statistics and sources instead of just agreeing with whatever they say. This gives it different evidence or an outside anchor to check against instead of being influenced by the user.

12. For issue 12, the design patterns that address latency are parallelism and routing. The 3 agents should run all at the same time to reduce time spent. Follow-up requests should use the synthesis agent and their information they have stored to avoid unnecessary calls. This reduces waiting rather than running every agent after another and rebuilding the report for every question.

13. For issue 13, the design patterns that address cost are caching and context compression. Reuse results that have been saved when the request and sources have not changed and summarize older turns to shed tokens. This reduces token use rather than repeating the same work over again and sending the full conversation every time.

14. For issue 14, the design patterns that address invalid formats are structured outputs and schema validation. Each agent should return all information using a JSON schema. Pydantic should validate the results. If it does not meet this required format, allow a fix to be requested. This is more dependable than asking for a format in the prompt because it doesn't mean it'll necessarily follow it the same every time.

15. For issue 15, the design pattern that addresses prompt sensitivity is a prompt template with systematic prompt evaluation. Test prompt changes on a small sample of opponents and check if they meet the goals and retrieve evidence that is relevant. Then keep changes only if they improve the results. This is better than accepting a rewrite just because it sounds right or detailed.

16. For issue 16, the design pattern that addresses information loss is structured handoff payloads. Each agent should summarize its findings in a format that has an opponent, weeks, evidence, and limitations when passing them on to another agent. This gives the synthesis agent the context it needs in order to summarize what was discussed because it can't see what the other agents saw.

17. For issue 17, the design pattern that addresses error propagation is validation between stages. Every result should be looked at as unverified, and it needs to be checked for its format as well as any evidence before passing it to the synthesis agent. This helps recognize mistakes before later stages, or agents carry them on.

18. For issue 18, the design pattern that addresses unbounded loops is defined termination criteria. The workflow should use a step cap, token budget, stall detection, and a partial answer is provided to degraded cases that says why it stopped. When a limit is reached it should stop further attempts and return a partial report only if allowed. This prevents repeated calls that make no progress. This is better than relying on the model to stop.

Final Thoughts

The hardest part of filling out this document was balancing the number of agents. Too few would stretch each agent across very different jobs, while too many would add latency, token cost, and handoff errors. We settled on four agents, each with a distinct job and toolset, and kept the calculations, formatting, and orchestration in deterministic Python with a different LLM call to verify claims. We believe this is a solid design and will perform the job that's necessary.